Classification and regression: constructing a neutral network

Sindre Øhman, Anders Mathias Rønneberg, Abdullah Ali Ahmed, Heidi Abrahamsen
(Dated: November 10, 2026)

Neural networks learn their own features from raw data, but this flexibility comes at the cost of many parameters and extensive hyperparameter tuning. The tipping point where this cost really pays off is therefore a central question. We designed a feed-forward neural network with back-propagation and applied it to two different types of problems; a regression problem, modeling the one-dimensional Runge function and classification of handwritten digit from the MNIST dataset.

[[RESULTATER]] On classification, a single hidden layer of 256 Leaky ReLU nodes reached a test accuracy of 98%, reducing the errors of the logistic regression baseline (92.6%) by 73.1%. We also proved that a second hidden layer gave no gain. [[KONKLUSJON 1 SETNINING]]

I. INTRODUCTION

The nature of the thinking human has puzzled the very humans who think since before our earliest written records. Around 400 CE, St. Augustine wrote in his *Confessions*:

> *I myself cannot grasp all that I am – the mind is too narrow to contain itself.*
>
> — St. Augustine, *Confessions* [1]

Sixteen centuries later, the rapid development of artificial intelligence (AI) has once again challenged our understanding of what a thinking mind is. In biological terms, thoughts arise from the activity of neurons in the brain, and as the mechanisms of these cells were uncovered, the pursuit to mimic them in a machine followed. McCulloch and Pitts [2] described the neuron as a simple threshold unit that either fires or stays silent, depending on whether the sum of its inputs exceeds a certain limit. Rosenblatt [3] built on this idea with the perceptron, in which the strengths of the connectors, the weights, could be learned from the data. Replacing the abrupt threshold with a smooth function such as the sigmoid made the network differentiable, so that its parameters could be adjusted systematically by gradient-based optimization. This is the foundation of the neural networks (NNs) in use today [4][5].

From this limited model of a single neuron, NNs have grown into tools that have transformed large parts of science and industry. A modern NN learns from data by adjusting a large number of parameters so as to minimize a loss function. Recent examples of what such models can achieve include the design of novel drugs [6] and a proposed solution of the roughly 80-year-old Erdős unit-distance problem [7][8]. Such results rely on models of enormous flexibility, and this flexibility comes at a price: vast numbers of parameters are needed in non-convex optimization problems with an array of hyperparameters to tune, and a large cost computational cost.

This raises the overarching question of this report. Classical methods such as linear regression rely on features chosen by hand, whereas a NN learns its own features directly from the raw data. When is the flexibility of a NN worth the cost, and when does a simpler model do just as well? Additionally, we will study how our NN tackles classification problems, including the special case of logistic regression. Based on this, we discuss what type of problem the NN is best suited for. Our ambition is therefore a modest one. We will not, any more than Augustine could, grasp all that we are. Instead we set out to grasp the neural network that was once modeled on our own biology.

For the regression part, we will fit the one-dimensional Runge function, comparing our network to linear regression models with corresponding penalties. We examine how the choice of architecture impacts the results; concretely, we vary the numbers of hidden layers and nodes, activation functions and penalty values. We also test different gradient descent methods, including stochastic gradient descent and the optimizers Adam and RMSprop. For the classification part, we classify the handwritten digits of the MNIST data set and compare our network with logistic regression. We compute the confusion matrix and discuss various metrics derived from this.

The report is structured as follows: Section II presents the theoretical background, first describing logistic regression as a baseline for neural networks, then describing the extension to neural networks. We assume a prior knowledge of linear regression and common methods of regression analysis such as cross-validation (CV), as well as gradient descent methods. Section III describes concretely what studies we have chosen to do, and the corresponding results are presented and discussed in Section IV. In Section V we conclude our result. The appendix includes a statement about sources of assistance that we have relied on, including LLMs. The code belonging to this report is available in a GitHub repository.

II. THEORY

A. The baseline: Logistic regression

Neural networks can be seen as an extension of logistic regression. Equivalently, we can see logistic regression as a special case of a neural network, where the number of hidden layers is zero. In order to understand NNs, it is therefore useful to start by describing logistic regres-

sion: This is a method of classification, which means that the logistic model enables us to determine the class of a sample. By class, we mean an output type, for instance whether a certain thing is identified (1) or not (0). It thus models a discrete output.

Logistic regression itself returns a probability – the probability that a certain sample belongs to a certain class. We say that an output y belongs to a class if $y = 1$. For a given class, the probability in question is then $p(y = 1|x)$, where x represents the inputs. The classification is only decided once a threshold is applied to this probability. The common choice is to divide the classes at $p = 0.5$.

There are two layers in a logistic regression model; the input layer and the output layer. The data set modeled by such a model will consist of a number of samples, with a number of inputs and a target output value for each sample. Equivalently, it may be formatted such that it has a number of outputs equal to the number of classes K, all of which take the value zero except for the target, which is 1. Binary problems are however an exception to the statement of having K outputs, as we commonly have a single output despite having two classes. This is simply because the two classes are the compliments of each other.

The first step in creating an appropriate logistic regression model is to form a linear combination of the inputs of a sample:

$$z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p = \mathbf{x}^T \boldsymbol{\theta}, \qquad (1)$$

where p is the number of inputs, θ_0 is called the bias, and the coefficients θ_i are called the weights – together they form the parameters $\boldsymbol{\theta}$. This is the same as in ordinary linear regression. The next step separates logistic regression from the latter: Since we want the model to output a probability, we need to map z to the domain $[0, 1]$. In binary logistic regression, the way we achieve this is to insert z into the logistic function, also known as sigmoid function,

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

For multinomial logistic regression where the number of classes exceeds two, we use another function – we will come back to this later. The sigmoid is commonly referred to as an S-shaped function, which transitions smoothly from 0 to 1 and takes the value $\sigma = 0.5$ at $z = 0$. Concretely, its tail goes to 0 as $z \to -\infty$ and to 1 as $z \to \infty$, and it is strictly bounded on the domain $(0, 1)$, meaning it may never return complete certainty. Now, one might ask why we do not simply use a step function instead; the main reason for this is that the sigmoid function, as opposed to a step function, is differentiable. Thus, it enables us to use gradient-based training.

This brings us to the next step, namely minimizing the loss function: To determine the weights and bias, we need to find the values most likely to produce the target output value. This is what maximum likelihood estimation (MLE) does. Equivalently, we can tag on a minus sign such that we instead get a minimization problem, which is preferable when we wish to use the standard gradient descent (GD) procedure. If we additionally take the natural logarithm of the likelihood, this "negative log-likelihood" is called the cross-entropy. This is a loss function that is equivalent to the MLE. The binary cross-entropy takes the form

$$\begin{aligned} C(\boldsymbol{\theta})_{CE} &= -\sum_{i=0}^{n-1} \left(y_i \ln p_i + (1 - y_i) \ln(1 - p_i)\right) \\ &= \sum_{i=0}^{n-1} \left(\ln(1 + e^{z_i}) - y_i z_i\right). \end{aligned} \qquad (2)$$

where we renamed the sigmoid function to σ to p, since it represents a probability. The pre-activation z represents the log-odds, and is connected to p via the logit transformation

$$z = \ln\left(\frac{p}{1 - p}\right).$$

Note that the weight parameters are located inside p_i – these cannot be extracted and solved for, meaning there is no closed-form solution.

The gradient of the binary cross-entropy can be found by taking the derivative with respect to a parameter component,

$$\begin{aligned} \frac{\partial C}{\partial \theta_j} &= \sum_{i=0}^{n-1} \left(\frac{\partial}{\partial z_i} \ln(1 - e^{z_i}) - y_i\right) \frac{\partial z_i}{\partial \theta_j} \\ &= \sum_{i=0}^{n-1} (p_i - y_i) x_{ij}, \end{aligned}$$

which extends to

$$\nabla_{\boldsymbol{\theta}} C = -\boldsymbol{X}^T (\boldsymbol{y} - \boldsymbol{p}). \qquad (3)$$

Examining this, we see that the gradient will get its largest contributions in cases where the model is confidently wrong, in which case $p_i - y_i$ will have absolute value 1. Thereafter, other incorrect results will have contributions between 0.5 and 1 depending on confidence. As for correctly classified samples, largest contributions are from cases where p_i is close to 0.5, i.e. for the most uncertain samples. Likewise, the very smallest contributions are from the cases where p_i is confidently correct, close to 0 or 1. In other words, the largest errors also get the largest adjustments, while the weights of the correct predictions are left undisturbed.

In cases of more than two classes, we use the softmax function instead of the sigmoid function:

$$p(C = k|\mathbf{x}) = \frac{e^{\mathbf{x}^T \boldsymbol{\theta}_k}}{\sum_{l=0}^{K-1} e^{\mathbf{x}^T \boldsymbol{\theta}_k}}, \qquad (4)$$

where $k = 0, \ldots, K-1$ denotes the class and the exponent is the pre-activation for a given output class, $\mathbf{x}^T\boldsymbol{\theta}_k = z_k$. As a side note, one may also recognize this expression as a probability from Boltzmann statistics (where the sigmoid function is the two-state system). The softmax function is used with the corresponding loss function, or multiclass cross entropy:

$$C(\boldsymbol{\theta})_{MCE} = -\sum_{i=0}^{n-1}\sum_{k=0}^{K-1} y_{ik} \ln p(C = k|\boldsymbol{x}_i) \tag{5}$$

where y_{ik} is the so-called one-hot encoding, which is equal to 1 if sample i belongs to class k and 0 otherwise, similar to a Krönecker delta-function. The corresponding gradient with respect to the parameters of a given class is

$$\frac{\partial C_{\text{MCE}}}{\partial \boldsymbol{\theta}_k} = -\boldsymbol{X}^T(\boldsymbol{y}_k - \boldsymbol{p}_k). \tag{6}$$

Logistic regression may also be simplified to linear regression if we simply omit the activation function, or equivalently replace it with the identity. In that case, the meaningful loss function will be the mean squared error (MSE), given by

$$C(\boldsymbol{\theta})_{\text{MSE}} = \frac{1}{n}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}\|^2, \tag{7}$$

with the derivative

$$\begin{aligned}\nabla_{\boldsymbol{\theta}} C_{\text{MSE}} &= \frac{1}{n}\frac{\partial}{\partial \boldsymbol{\theta}}\left[(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta})^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta})\right] \\ &= \frac{2}{n}\boldsymbol{X}^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\theta}).\end{aligned} \tag{8}$$

B. The extension: Neural networks

Neural networks can be seen as logistic regression with the addition of hidden layers. While logistic regression only has an input layer and an output layer, neural networks have intermediate layers in between, which consist of a number of nodes which we call neurons. Each of these hidden layers has its own "feature matrix" which we call the activation matrix A, with its own features. Continuing, each connection between neurons then has its own weights. For a network with many hidden layers, the number of parameters will then be enormous – especially for a fully connected network where all neurons of adjacent layers are connected, which is the case that we will consider in this report. The computational cost will then grow accordingly. The reason that we still employ this structure is that this gives NNs a unique property: Unlike classical regression methods, NNs are able to learn not only weight parameters of the features, but also the features themselves. In other words, we do not choose the features manually beforehand – the brilliance of NNs is that the features are *learned* as the model develops in training.

The structure of a hidden layer is more or less the same as the output layer in logistic regression. In the latter case, we had a number of inputs for each output node; these formed the pre-activation z, which we inserted into the sigmoid function σ (or the softmax) giving the output probability $p = \sigma(z)$. For a hidden layer, the procedure is the same. The input of a given neuron comes from the previous layer, and the output returned becomes the input to the subsequent layer. However, the function transforming the pre-activation in the hidden layers does not have to be the sigmoid or softmax function; instead, it can be any general activation function.

1. *Activation functions*

Originally, machine learning used the sigmoid function, or logistic function, as the activation function of the neurons. As mentioned, the sigmoid function had the key property of differentiability. We observe that this derivative is largest when $z = 0$, which gives $\sigma' = 1/4$. In other words, $\sigma' \leq 1/4\ \forall z$. This causes a problem; for each layer, the gradients are multiplied with a factor $\leq 1/4$, such that the gradients quickly vanish. This means that the network essentially stops learning, a result that kept the field of machine learning stagnated for two decades. [9] The advancement of the field continued when an alternative activation function was introduced that avoided this problem. This was the ReLU function,

$$\text{ReLU}(z) = \max(0, z)$$

which has the derivative

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases} \tag{9}$$

where the derivative at $z = 0$ is undefined, but in practice set to 0. So unlike the sigmoid, ReLU doesn't shrink the gradients. However, the ReLU function has another problem: If the pre-activation z of a neuron is negative for all training samples, then that neuron will "die" – the ReLU will evaluate to 0 such that its gradient is also 0, and the weights stop updating. The neuron will be dead for the remaining of the training. We can avoid this from happening by instead using the so-called leaky ReLU, defined as

$$\text{LReLU}(z) = \begin{cases} z, & z > 0 \\ \alpha z, & z < 0 \end{cases}$$

where α is some small fraction that commonly takes the value 0.01. The derivative is then instead

$$\text{LReLU}'(z) = \begin{cases} 1, & z > 0 \\ \alpha, & z < 0, \end{cases} \tag{10}$$

such that the gradients will never vanish completely.

Note that although the hidden layers may use any activation function, the output layer still needs to use the Sigmoid function for a binary problem, and the softmax for a multiclass problem. The loss function needs to correspond to this activation function as well; we use the binary cross-entropy with the Sigmoid, the softmax loss with the softmax function and the MSE with the identity. These are pairs that have been tried and tested, and are known to be compatible.

2. Separability and regularization

It is both useful and standard practice to introduce regularization terms to the loss function. Adding an L_1 or L_2 penalty to the loss function, Eq. (2), (5) or (7), corresponds to adding the term $\lambda\|\boldsymbol{\theta}\|$ or $\lambda\|\boldsymbol{\theta}\|^2$, respectively. Doing so then results in the respective gradient, Eq. (3), (6) or (8), being extended with the term $\lambda\,\mathrm{sgn}(\boldsymbol{\theta})$ or $2\lambda\boldsymbol{\theta}$.

The reason that we need regularization is connected to what we call separability: Classes are separable if there exists a decision boundary such that every sample is correctly classified. A decision boundary is the boundary separating predicted classes in feature space. For logistic regression, it is linear. For multiclass logistic regression, the decision boundaries form a hyperplane. If a problem is such that a linear graph or hyperplane may actually separate the classes perfectly, it is separable. For neural networks, the decision boundaries are non-linear, such that a model of sufficient complexity and well-chosen activation functions can in principle make any problem separable given that we have a finite data set with deterministic labels. This also follows from the universal approximation theorem: It states that a FFNN with one hidden layer is a universal approximator for continuous functions on compact domains if and only if the activation function is not a polynomial on any interval. With that said, whether the training is able to recover this model is an entirely different question, and comprises the challenge of machine learning.

The problem with separability is that the model does not know when to stop: When every point is correctly classified, increasing the weight will keep increasing the certainty of the predictions and improving the loss. Therefore, the model will continue training even after every sample is correctly classified, increasing the certainty of the probabilities towards 0 or 1, while the loss approaches 0, without ever reaching it. Hence, we get an overconfident model and indefinitely growing parameter values. Regularization prevents this from happening in introducing a penalty that grows according to parameter size. This keeps the solution finite and retains meaningful probabilities. For neural networks, it is then especially important in cases where the model is complex enough to achieve separability.

3. Backpropagation

As mentioned, we use gradient descent to train the network, i.e. update the weights and bias. The way the gradient is calculated is through what we call backpropagation, or reverse mode. This consists of starting from the output and using the chain rule with respect to each weight, going backwards all the way to the weights of the input. We reuse partial derivatives such that in the when moving on to the next (previous) layer, only one new partial derivative has to be computed.The details of this can be found in our GitHub repository. Alternatively, we could compute the gradient in forward mode, starting from the input and working our way towards the output. Machine learning, however, relies on backpropagation. The reason for this is that the number of inputs is far greater than the outputs. Therefore, we also require fewer passes through the data if we start from the output; whereas if we went the other way, we would need to do more passes, such that we recompute several derivatives and do unnecessary work in that sense. Note that if the number of outputs exceeded the inputs, the forward mode would instead be the cheaper choice.

4. Weight initialization

To ensure stable training and mitigate the vanishing and exploding gradient problems, it is important to initialize the weights such that the variance of activations and gradients remains approximately constant throughout the network. This can be achieved by sampling the initial weights from a zero-mean Gaussian distribution and scaling the variance according to the number of neurons in each layer. The commonly used initialization method for the Sigmoid function is the Xavier initialization:

$$W_{\text{Xavier}} \sim \mathcal{N}(0,1)\sqrt{\frac{1}{n_{\text{in}}}}, \tag{11}$$

where n_{in} is the number of inputs for a given neuron, and W is a matrix containing the initial weights of a layer. The ReLU activation family sets all negative inputs to zero, and effectively reduces the variance of activations by approximately half. It is therefore more common for the ReLU-family to use He initialization, given by

$$W_{\text{He}} \sim \mathcal{N}(0,1)\sqrt{\frac{2}{n_{\text{in}}}}, \tag{12}$$

which has a factor 2 in the variance compared to Xavier.

III. METHODS

1. Development of the Neural Network

We developed a fully connected FFNN with a flexible number of hidden layers and neurons, changeable acti-

vation functions and cost functions and an optional L_1 or L_2 regularization. For the training, we implemented stochastic gradient (SGD) with both momentum and plain gradient descent, as well as three adaptive methods; Adagrad, RMSprop and Adam. The weights of the FFNN were initialized using He initialization for layers employing ReLU-family activation functions, while Xavier initialization was used for all other layers.

To verify the correctness of our implementation, we tested the FFNN against established neural network frameworks, including JAX and Scikit-learn. The gradients computed by our implementation were compared against those obtained using JAX's automatic differentiation. Furthermore, we compared our NN to Scikit-Learn's NN implementation by fitting both models to the Runge function.

2. *Regression*

Our first main objective was to study how our NN would perform in a concrete regression problem, and how it would compare to linear regression. We chose to model the one-dimensional Runge function,

$$f(x) = \frac{1}{1+25x^2},$$

in the interval $x \in [-1, 1]$. The data set was then generated uniformly from this function, where we added a normally distributed noise term $\epsilon \sim \mathcal{N}(0, \sigma^2)$ with standard deviation $\sigma = 0.1$. The number of samples was set to $n = 100$ throughout the regression part of the project, and in the NN we use the identity as activation for the output layer. In cross-validation assessments we have consistently used $n \cdot 4/5 = 80$ data points for our training set, implemented by means of Scikit-learn's function `train_test_split` [10].

We initially wished to compare our NN with linear regression using ordinary least squares (OLS). The comparison was between a polynomial OLS model, whose polynomial degree was chosen by cross-validation, and two different NN; one with a single hidden layer of 50 neurons, the other with two hidden layers consisting of 100 neurons each. Both NNs used the Sigmoid function as activation function in the hidden layers. The results were plotted as graphs showing the cross-validated MSE as well as the training MSE, both as a function of gradient descent epochs. The OLS parameters were found using the closed-form solution.

Additionally, we did the same analysis where we instead used SGD and tested different optimization methods. We compared plain GD to the adaptive methods RMSprop and Adam, testing the effect of using different batch sizes 1, 5 and 10 in each case, in addition to the fullbatch version. When doing this, we also tested different learning rates, plotting the cross-validated MSE as a function of learning rate. Based on these results, we chose a GD method and a batch size for each of the two NNs, which we used throughout the remainder of our investigations. The decision was based off of overall performance, since the learning rate would be decided at a later point in combination with the regularization. All plots were made using Matplotlib [11].

Subsequently, we investigated how the choice of activation function impacted our predictions. We compared three different activation functions for the hidden layers; the Sigmoid that we had been using so far, and the ReLU and Leaky ReLU functions. For each version, we investigated the influence on performance as a function of both width (number of neurons) and depth (number of layers). We plotted the error as a function of hidden layers from 1-5, where the number of neurons per layer was held fixed at 50, and the error as a function of neurons from 1-100, where the number of hidden layers was fixed to 1. Based on this, we chose an activation function to carry on with. We also discussed whether there were signs of overfitting.

Lastly, we studied the effect of regularization. As before, we compared our NN results to those of the corresponding linear regression method. We compared the L_2 norm with Ridge, and the L_1 norm with Lasso. The error was plotted as a function epochs for both of the NNs we have been using (one layer of 50 neurons and 2 layers of 100 neurons each). For each of these, we chose a combination of learning rate and penalty though a cross-validation grid search. The decision was made by choosing the method or value giving the lowest cross-validated MSE, and then applying the 1 standard error (1SE) rule for the hyperparameters. The 1SE rule states that one should choose the value giving the simplest model within one standard error of the value giving the lowest error. The simplest model here means the highest regularization and the largest learning rate.

3. *Classification*

Our other objective was to study how the NN performs in a classification problem. To test this, we used the MNIST dataset [12]. It consists of 70 000 pictures of handwritten digits, split into a training set of 60 000 and a test set of 10 000 examples. Each image is 28 x 28 = 784 pixels. Figure 1 is a showcase of random samples from the dataset.

Because a FFNN takes a vector as input, each image was converted to a vector of the 784 pixel values. The values, integers between 0 and 255, were scaled to $[0, 1]$ by dividing by the maximum value. This was done to optimize the conditions for gradient descent. The class labels were encoded to fit the one-hot format, making them 10-dimensional vectors. This was done in order to match the softmax output, so that we could calculate the loss function of Eq. (5). 10 000 of the training data were split off to a validation set, used in hold-out grid searches. The test set was only used once in testing the final NN.

We made three different models to study the perfor-

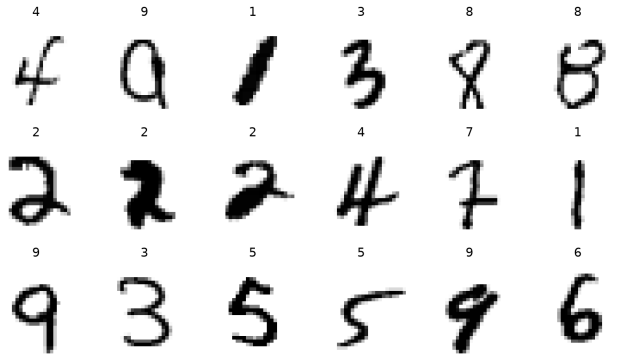


FIG. 1. Examples of the handwritten digits that constitute the MNIST-dataset, with their respective targets above. One picture consists of 28 x 28 pixels. [12]

mance of the NNs. Firstly, we used a NN with no hidden layers, in other words a logistic regression model. Next, we chose one NN with a single hidden layer and one NN of 2 hidden layers, both having 256 neurons per layer. Each of these were trained in the same way, with Adam as optimizer, using a minibatch size of 100 samples and training over 10 epochs.

The choice of hyperparameters, including activation functions, was made through hold-out grid search. With this, one model is trained for each hyperparameter of table I, and tested with the validation set. We used a two-dimensional grid such that the hyperparameters were tuned in pairs: These were γ and λ, width and depth, and activation functions and width. For each pair, the performance of the corresponding network on both the training set and the validation set was visualized in heatmaps, using the `seaborn` visualizer package [13]. The hyperparameter values were chosen based on the validation data, choosing the pairs with the highest prediction accuracy. The values that were included in the grid search are presented in Table I. We chose this method over cross-validation grid search, because the latter demands a higher computational cost. With 60000 samples in the training set, where 10000 are used to validate, we regard the hold-out method as stable enough.

To verify our implementation, we compared our best network and our logistic regression baseline with the corresponding models of Scikit-Learn. The network was compared with `MLPClassifier`, configured to match our own network as closely as possible.

Once the hyperparameters were determined, we analyzed our classification model with different performance measures. Firstly, we constructed a confusion matrix showing the outcome from the classification for all k classes. Here the element C_{ij} is the number of samples with true class i that was predicted as j. Thus, the diagonal elements are correctly classified, or true positive (TP_k). For a given class k, the off-diagonal elements in row k are its false negatives (FN_k), and the off-diagonal elements of column k are false positives (FP_k). All other

TABLE I. Hyperparameter values used in the pairwise grid searches.

Hyperparameter	Values
Learning rate γ	10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}
Regularization λ	0, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}
Width (nodes per layer)	16, 32, 64, 128, 256
Depth (hidden layers)	1, 2, 3
Activation function	`sigmoid`, `ReLU`, `Leaky ReLU`

elements in the matrix are true negatives (TN_k). From these we can derive the following evaluation metrics:

$$\text{Recall}_k = \frac{TP_k}{TP_k + FN_k} \tag{13}$$

which shows the true positive rate, i.e. what ratio of the positive samples were correctly identified. The precision

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k} \tag{14}$$

which gives the positive predictive value, i.e. what ratio of the positive predicitons were correct. The F_1 score, which is the harmonic mean of the two former

$$F_{1,k} = 2 \cdot \frac{\text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} \tag{15}$$

which is small if either the precision or recall is small, and finally the accuracy, given by

$$\text{Accuracy} = \frac{1}{n} \sum_k TP_k \tag{16}$$

where n is the total number of samples, showing the overall rate of correctly identified samples per class.

We also reported the results of setting all predictions equal to the majority class, the most commonly predicted class.

IV. RESULTS AND DISCUSSION

The gradient computation done by our NN implementation was verified against JAX's automatic differentiation with differences smaller than machine precision. When comparing our network to that of of Scikit-Learn, the resulting models showed very similar behavior in terms of MSE values, with only minor differences attributable to variations in weight initialization.

A. Regression

The comparison between OLS, a NN with a single 50-neuron hidden layer and a NN with two 100-neuron hidden layers is shown in Figure 2. Keep in mind that these

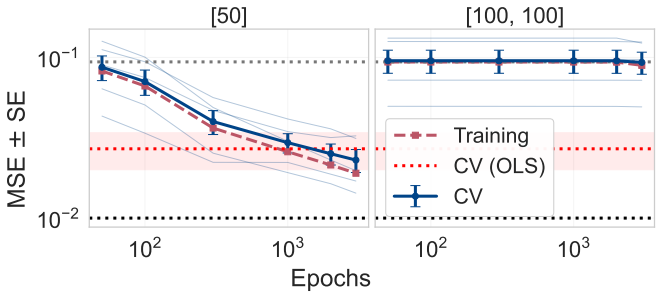


FIG. 2. MSE as a function of epochs for two different configurations of the NNs depth and width, left figure with a single 50-neuron hidden layer and the right with two 100-neuron hidden layers. Shows an untuned NN with learning rate $\gamma = 0.1$ trained on epochs [50, 100, 300, 1000, 2000, 3000]. The dark blue line shows 5-fold CV MSE, with corresponding individual folds in light blue. The training MSE is shown by the dark red graph with circular markers. The red horizontal line shows the CV MSE for OLS estimated in the same way. The upper dotted horizontal line indicates the variance in observations $\text{var}(y) = 0.94$, and the lower black line indicates the noise floor $\sigma^2 = 0.01$.

are untuned networks, i.e. we have not gone through any hyperparameter selections at this point. We see that the single-layer network achieves a smaller MSE as we increase the number of epochs, while the two-layer network has little improvement. The single-layer network barely manages to surpass the OLS solution, if accounting for the standard error, at 3000 epochs. On the other hand, it has a far greater computational cost, and a number of parameters consisting of $1 \cdot 50 + 50 \cdot 1 = 100$ weights and $50 + 50 = 100$ biases, giving 200 parameters per sample, whereas linear regression only has 6, which is the number of polynomial features found to be optimal from cross-validation. Thus, there seems to be no advantage of using untuned NNs as opposed to a simple polynomial model for a typical regression problem.

In Figure 3 we see how optimization methods RMSprop and Adam compare to plain GD for each of the two NNs, for varying minibatch-sizes and learning rates. The upper row shows the single-layer case, the lower the two-layer case. For plain GD, we observe that SGD significantly reduces the error with smaller learning rates. Since we use the same number of epochs for all data points in the graphs, this pattern is expected, as small learning rates likely do not manage to progress much. Therefore, SGD, which multiplies the number of steps per epoch, makes convergence faster and is especially beneficial. In accordance with this, the benefit is largest for the smallest minibatch size, giving the most steps, and the benefit shrinks as the learning rate grows larger. For the adaptive methods, on the other hand, we observe that SGD has less of an impact. This is a reasonable result, since SGD can make the moment-estimates of the adaptive methods less precise. Unlike plain gradient descent, whose learning rate is fixed, the adaptive methods have a time-dependent learning rate individual to each gradient component. Therefore, they did in this case not benefit from minibatches in the same way, but instead performed better with the maximum precision of the full gradient. However, the minibatch size 10 model overall performed similarly to the fullbatch model for the smaller network.

Out of the different GD methods, we see that Adam overall performs the best across different learning rates. Since the learning rate was to be decided later in a grid search together with regularization, we chose to employ Adam for the remainder of our regression studies. We did not make a decision about the architecture of the network yet, although the larger network seemed to have somewhat larger errors overall. This was because we still had not introduced regularization, thus we did not want to rule out the network just yet. We did, however, also make a decision about the minibatch size at this point. The fullbatch version was chosen for the two-layer network, which seemed to clearly outperform the SGD models, while we chose batch size 10 for the single-layer network, as it seemed to perform equally well with this size as with the full gradient, and thus we chose the cheaper model.

In Figure 6, we study the effects of using different activation functions in our hidden layers as a function of depth and width (number of layers and neurons). We see that the Sigmoid has a lower error when the number of layers is smaller – although not shown in the figure, the value was confirmed to stay consistently high as the number of layers increased – indicating the characteristic vanishing of gradients. ReLU, on the other hand, does not have this issue. However, at about layer 6 we observe that the training error continues to fall whereas the cross-validation error has a slight increase. This points to overfitting. As for Leaky ReLU, we see that the error relatively speaking explodes at specifically layer 5. One contributing reason for this could be the fact that we used the same weight initialization for Leaky ReLU as for ReLU, while it is more common to instead adapt the He initialization to include the tiny α^2 factor in the denominator of the variance in Eq. 12. However, most likely using 5 layers happens to align certain structure causing this effect. In any case, we assert that Leaky ReLU is unstable for our model, and that ReLU does not show any signs of "dying" neurons for the ranges we will be working with. Thus, we chose to continue with ReLU as our hidden layer activation function. As a function of neurons, we see that both the training and the CV error rise around 200 neurons. This indicates that a structurally, a high number of neurons inhibits successful training for our problem – it does not indicate overfitting, since the training also increases. Looking at the errors, we note that a single hidden layer seemed to be sufficient for performance, and that 50 neurons gave the lowest error, however when we take the standard error into account, only 10 neurons might perform equally well.

At this point, we started introducing regularization. We first determined the optimal penalty together with

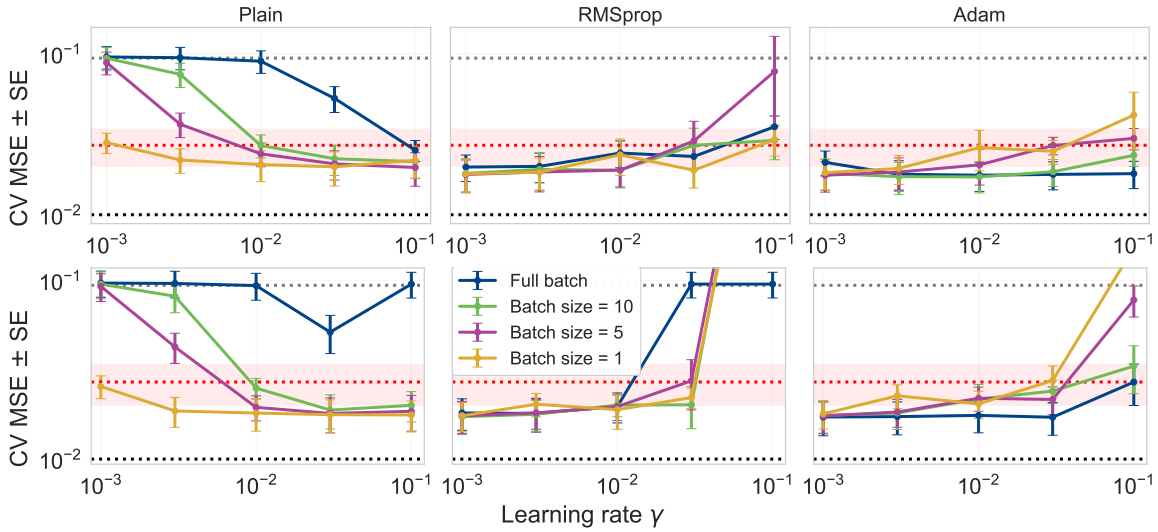


FIG. 3. Cross-validated MSE as a function of learning rate for fullbatch and SGD models with batch sizes 10, 5 and 1, using different optimization methods. Each datapoint has 2000 epochs of training. Upper row shows an NN with one hidden layer with [50] nodes, and lower row showing for two hidden layers [100, 100]. The remaining graphs are identical to Figure 2.

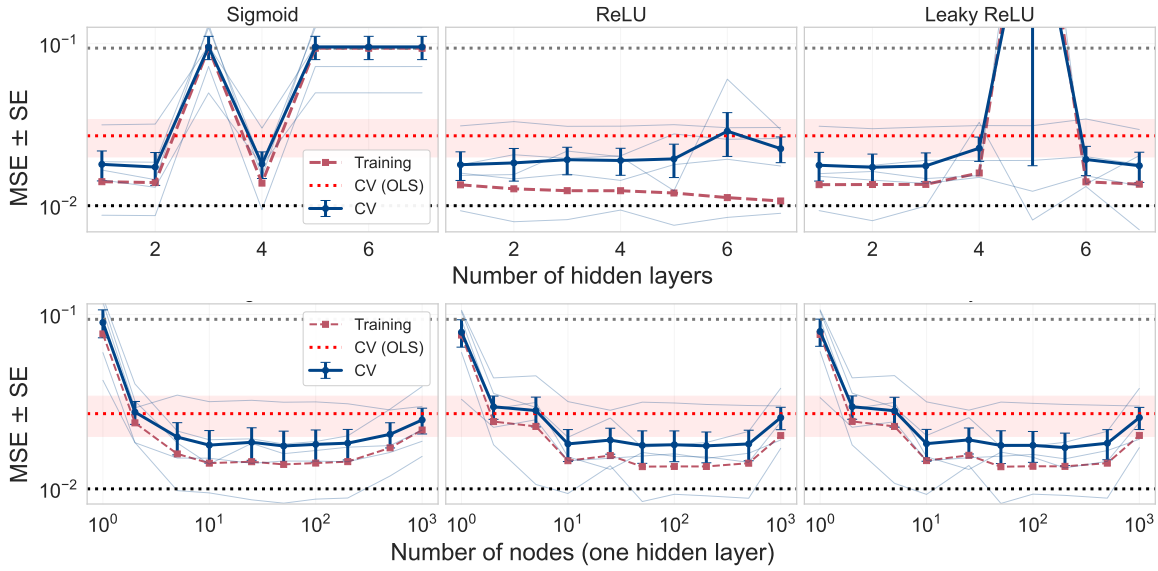


FIG. 4. Upper row shows cross-validated MSE as a function of hidden layers with 100 nodes per layer. Lower row shows the error as a function of number of nodes of a single hidden layer. Each column displays a different activation function for the hidden layers. The learning rate is $\gamma = 0.03$ and the number of epochs 300. Each graph represents the same quantities as in Figure 2.

learning rate in a CV grid-search; this is shown in the heatmap of Figure 5 for an L2 norm. The L1 norm has very similar values.

B. Classification

Accuracy values derived from the grid search are plotted in the heatmap of figure 7. Accuracy calculated from the validation set is the top number, and in parenthesis the values calculated from the training set. The accu-

TABLE II. CV results for OLS, Ridge, and Lasso, estimated using the one standard error rule for polynomial degree and hyperparameter λ. Estimated using 5-fold CV on $n = 80$ training data points.

Method	CV MSE ± SE	Degree	λ	Mean fit time [s]
OLS	0.027 ± 0.007	8	-	0.00059
Ridge	0.027 ± 0.006	8	1×10^{-5}	0.00064
Lasso	0.029 ± 0.004	6	0.001	0.01013

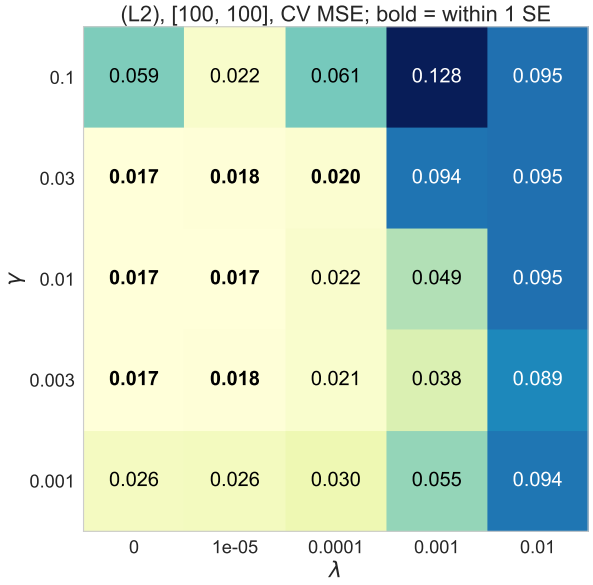


FIG. 5. CV grid search of learning rate γ and penalty λ for NN with L2 regularization and two hidden layers [100, 100].

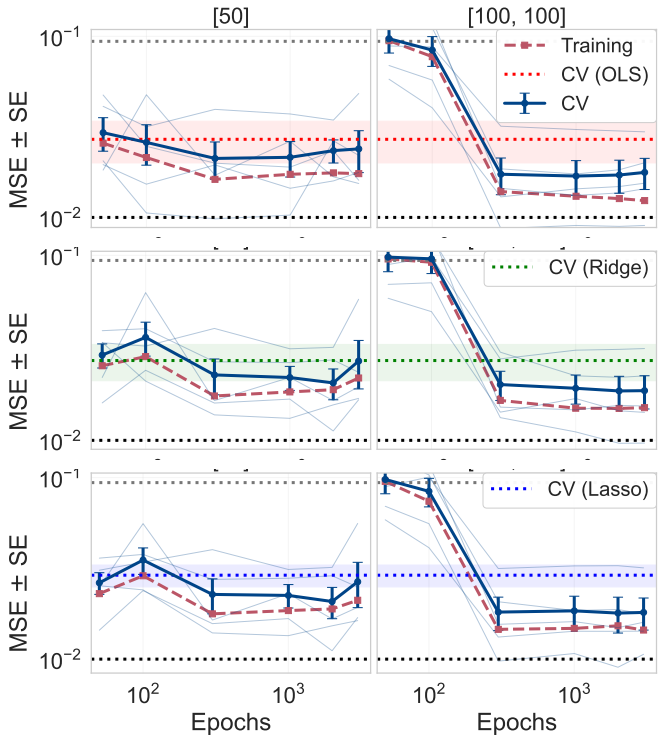


FIG. 6. MSE as a function of epochs=[50, 100, 300, 1000, 2000, 3000] for two different configurations of hidden layers [50] and [100, 100] for a tuned NN. Blue line indicating the 5-fold CV MSE estimated using $n = 80$ data point with corresponding training MSE shown by square red marker. The rows from top to bottom, show unpenalized NN together with CV for OLS, L2 penalized NN with CV Ridge, and L1 penalized with CV Lasso indicated by colored horizontal lines. Upper stippled horizontal line indicating the variance in observations var(y) = 0.94 and the lower black line indicating the noise floor $\sigma^2 = 0.01$.

TABLE III. Final CV results for NN. Estimated using 5-fold CV on $n = 80$ training data points.

Penalty	CV MSE ± SE	γ	λ	Epochs	Batch size	t[s]
1 layer, 50 nodes: [50]						
-	0.022 ± 0.005	0.1	-	300	10	0.08
L2	0.023 ± 0.005	0.1	0.0001	300	10	0.08
L1	0.023 ± 0.005	0.1	0.001	300	10	0.08
2 layers, 100 nodes per layer: [100, 100]						
-	0.017 ± 0.004	0.03	-	300	fullbatch	0.10
L2	0.020 ± 0.004	0.03	0.0001	300	fullbatch	0.10
L1	0.017 ± 0.004	0.03	0.0001	300	fullbatch	0.11

TABLE IV. Final test MSE results for NN and Regression methods. Calculated using $n = 20$ test data points.

Model	test MSE ± SE	Parameters
OLS	0.019 ± 0.005	8
Ridge	0.018 ± 0.005	8
Lasso	0.023 ± 0.009	6
[50]		
NN	0.018 ± 0.006	151
NN L2	0.020 ± 0.008	151
NN L1	0.021 ± 0.008	151
[100, 100]		
NN	0.013 ± 0.005	10401
NN L2	0.018 ± 0.006	10401
NN L1	0.016 ± 0.005	10401

racy of the validation run gives color to the cell, higher value gives a deeper blue. In figure (a) the model with learning rate $\gamma = 10^{-3}$ gains the highest accuracy. The regularization parameter appears to have a wider optimum, as the values of $\lambda = 0$ and $\lambda = 10^{-5}$ have the same accuracy and $\lambda = 10^{-4}$ is close with its difference of only 0.002. This signalizes that our choices of lambda values for the grid search most likely covers the best accuracy. The accuracy of the learning rate, however, is highly sensitive around $\gamma = 10^{-3}$, with large differences between neighboring cells. Moving one step above or below this value causes a a sharp drop in accuracy. Judging from this there could be a better value of γ within the interval $\{10^{-2}, 10^{-3}\}$ which would be revealed through a new grid search of narrower intervals. This applies to most of the heatmaps of figure 7. We choose $\lambda = 10^{-5}$ rather than $\lambda = 0$ for the logistic regression model, seeing as a small penalty stabilizes the model better than no penalty does. In (b), λ has little effect at the optimal learning rate $\gamma = 10^{-3}$, where $\lambda = 0$, 10^{-5} and 10^{-4} all give a validation accuracy of 0.975. However, $\lambda = 10^{-4}$ slightly reduces the training accuracy from 0.993 to 0.989 without any loss in validation accuracy, indicating a small reduction in overfitting. The minor penalties also makes the model more robust. At $\gamma = 0.1$ the unregularized model reaches only 0.750 (training: 0.758) compared to 0.852 for $\lambda = 10^{-5}$, due to the penalty keeping the weights from growing too large. This is why, in the choice of the three optimal lambdas avoid $\lambda = 0$ and uses $\lambda = 10^{-4}$ to tune the model. (c) is the heatmap from grid search over

TABLE V. Optimal values of hyperparameters derived from grid search, with values of table I.

Parameter	λ	γ	Width	Depth	Act function
Optimal value NN	10^{-4}	10^{-3}	256	1	`Leaky ReLU`
Optimal value Log.Reg.	10^{-5}	10^{-3}	-	-	-

depth against width. In the top and bottom right corners of the graph are the optimal accuracies, both at 0.979. In the choice between a depth of 1 and 3, 1 is the better alternative. Simply because this saves a big amount of processing power for the same accuracy. The accuracies increases with width until the highest handpicked value. This indicates that the interval the grid searches over is too low, and does not encompass the optimal.

In the last heatmap of figure 7, (d), the three activation functions `sigmoid`, `ReLu` and `Leaky ReLU` are plotted against width. `ReLU` and `Leaky ReLU` are nearly identical in performance, due to their near identical definitions. `Leaky ReLU`, however, avoids the problem of dead neurons discussed in Sec. II B 1. Of this reason, we chose the `Leaky ReLU` as the activation function of the hidden layer of our classification NN. The activation function of the output layer is always `softmax`. The optimal hyperparameters resulting from our grid search are presented in table V.

Figure 8 presents the confusion matrix, with the true value of the drawn digit on the y-axis and the predicted digit on the x-axis. The right-hand side bar illustrates how the color changes from light green to dark blue as the number of wrong predictions increases. As the figures shows, the most frequent mistake is predicting the digit 9 as 4 ($9 \rightarrow 4$) with 13 incidences. $2 \rightarrow 8$ occurred 12 times, and $7 \rightarrow 2$ 8 times. The mistakes are reasonable, as the mistaken digits all are similar in shape.

The contents of the confusion matrix are further described by the barplot of figure 9. Here, precision (blue), recall (light green) and their harmonic mean F1 scores (red) is plotted for each digit. These metrics are a more direct illustration of how the model's error affect each class. In the case of class 9, the most frequent error ($9\rightarrow$ 4) is a false negative. This decreases the metric of recall, according to Eq.13. For the class 4, however, this same mistake does not affect the recall. Rather, it is the metric precision that is affected (Eq.14). The class of 2 is another example of good precision, low recall. This is reflected in the confusion matrix as a high number of errors in the true digit row of 2 (false negatives), while the number of errors in the column of predicted 2's is lower (false positives). This underlines the necessity to evaluate the predictions of a network from multiple angles. As such, the harmonic mean F1 serves as a balanced metric between the two (Eq.15. Judeging from F1, our model gives the best results predicting the simple structures of 0 and 1, while being worse at the digits 2, 8 and 9.

Table VI summarizes the number of parameters, errors and test accuracy for our model generated at tree different depth, $0, 1$ and 2. The table also contains equivalent models built with Scikit-learn for depth of 0 and 1. Seeing as this is a widely used and well tested library in machine learning procedures, the models of Scikit-learn are used as a reference for our own implementations in this report [10]. Beginning with the logistic model with no hidden layers, the table shows our model performs fine. Compared to the Scikit-learn model, we produce a lower amount of errors (739 against 743), and an accuracy that is better with 0.04 percentage points ($0.9261 - 0.9257$). These are subtle differences, deriving directly from subtle differences in the model. These are discussed in Appendix B. While still being subtle, the differences in performance increase in favor of scikit-learn when one hidden layer is introduced. With this, our model gives 9 more errors, and produces an accuracy 0.09 percentage points below the scikit-learn model ($0.9810 - 0.9801$). This is within one standard error of Scikit's model (± 0.0014) . In this report we implemented scikit-learns `MLPClassifier` with the same architecture, loss, optimizer, batch size, epoch and L2 penalty as our model. The models differ in activation (`Leaky ReLU` vs `ReLU`), weight inizialization (He vs Glorot-uniform) and random number generator. Considering these differences, a deviation of 9 errors are to be expected [10]. In comparison with the Scikit-standard models, our NN and Logistic regression model performed very well. With our model performance validated against the standard, we claim that our one layer NN performed better than the baseline of logistic regression. As proven by the parameter column of table VI, the NN is far more expensive in computational power. Luckily, this is justified with a good result. The errors made are reduced by 73.1 % (199 errors against 739). Lastly, we designed a NN with one additional hidden layer to illustrate that the number of parameters is not proportional with increase in performance. In our example, the additional hidden layer of width = 256 and with activation function `Leaky ReLU` gave an increase of 65 792 parameters, while also increasing the number of errors from 199 to 215. The difference is about one standard error (0.98010.0014 vs. 0.97850.0015), indicating no significant change in performance. We keep in mind that the hyperparameters were tuned for one layer. The extra parameters might also contribute negatively if they overfit the data.

To put the accuracies into perspective, regard Table C1 and the distribution of digits in the MNIST training and test set. With each digit making up roughly 10% of the data, the classes are nearly balanced. The class with the most samples is digit 1, with its 11.35% of the test set. The simplest possible classifier, predicting that every sample belongs to the majority class, would thus obtain an accuracy of 0.1135. Logistic regression raises this to 0.9261 and our NN removes a further 73.1% resulting in an accuracy of 0.9801. With such balanced data, the errors of Figure 8 can not be explained by class frequency. The least frequent class (digit 5) is classified well, while the classes of highest error (2,8 and 9) are of average

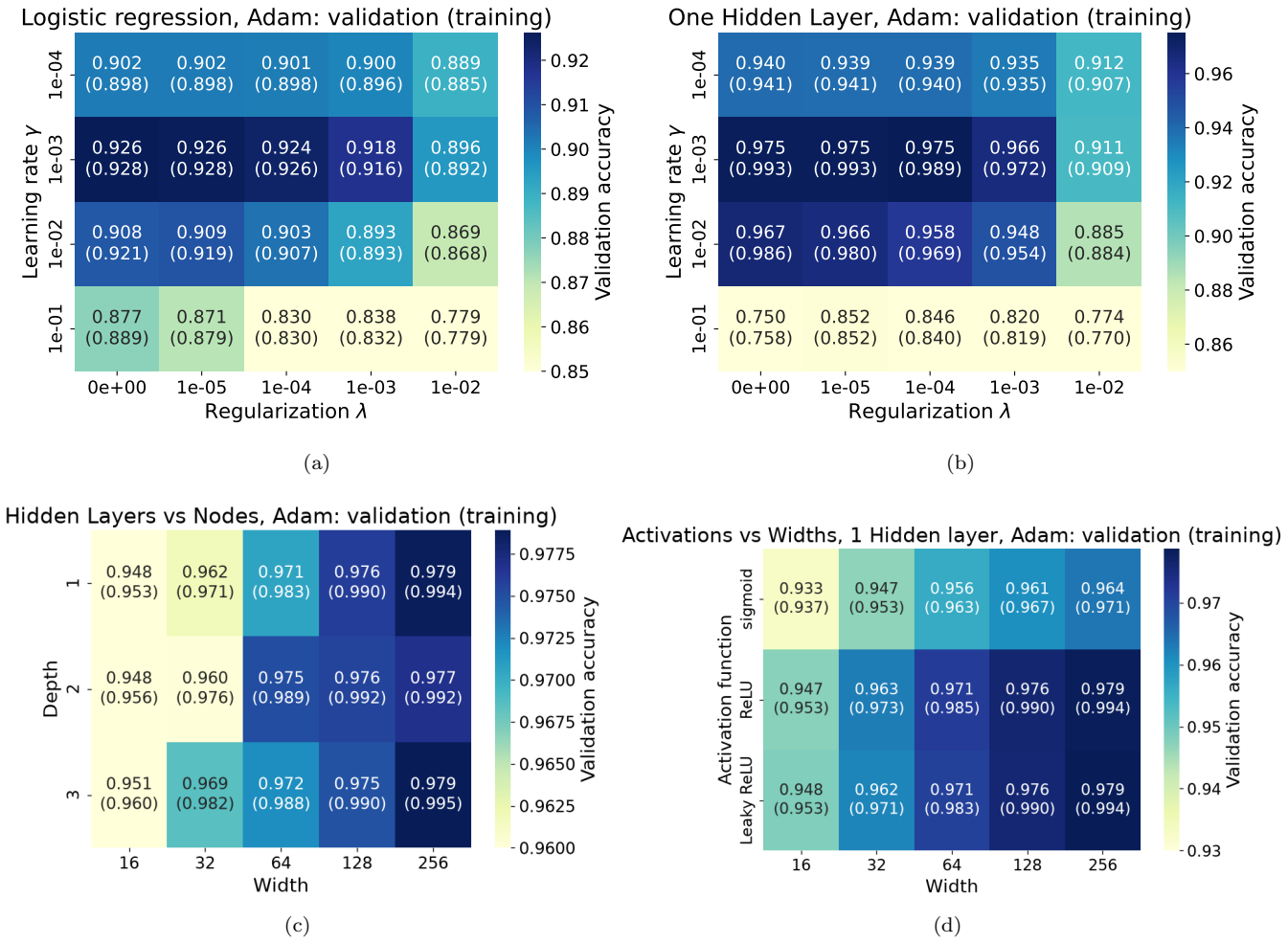


FIG. 7. Heatmaps generated from grid search with values from Table I. Color and number in each cell reflects accuracy from validation data. Accuracy from the training set is in parenthesis below. All models use Adam as optimization. (a) Learning rate γ against regularization value λ, in logistic regression model. (b) Learning rate γ against regularization value λ, in NN with one hidden layer. (c) Depth against width. (d) Activation function against width, in NN with one hidden layer.

TABLE VI. Number of parameters, errors and test accuracy for the different models on the MNIST test set ($n = 10\,000$). Scikit-Learn above the line, our own implementation below. All neural network models have width 256.

Model	**Parameters**	**Errors**	**Accuracy**
Logistic (Sklearn)	7850	743	0.9257 ± 0.0026
1 layer, ReLU (Sklearn)	203530	190	0.9810 ± 0.0014
Logistic	7850	739	0.9261 ± 0.0026
1 layer, Leaky ReLU	203530	199	0.9801 ± 0.0014
2 layers, Leaky ReLU	269322	215	0.9785 ± 0.0015

V. CONCLUSION

... For the classification of the MIST dataset, logistic regression reached a test accuracy of 92.6%. A network with one hidden layer of 256 Leaky ReLU nodes reached 98.0% cutting the number of errors by 73%. This accuracy is within one standard error of Scikit-learns corresponding network, supporting our implementation. The remainding mistakes cannot be explained by class imbalance, since the samples are nearly balanced.

frequency.

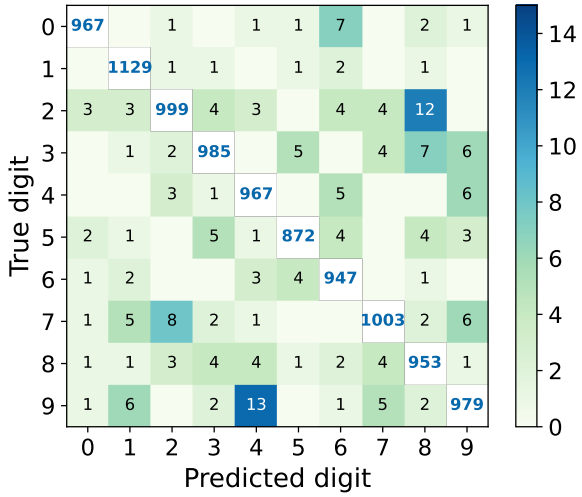


FIG. 8. Confusion matrix of results produced by our best performing model (Hyperparameters in table V), evaluated on the MNIST dataset. Rows show true digit, columns show predicted digit. Diagonal elements are the correctly classified samples, and off-diagonals are misclassifications. Color scale deepen with the number of errors.

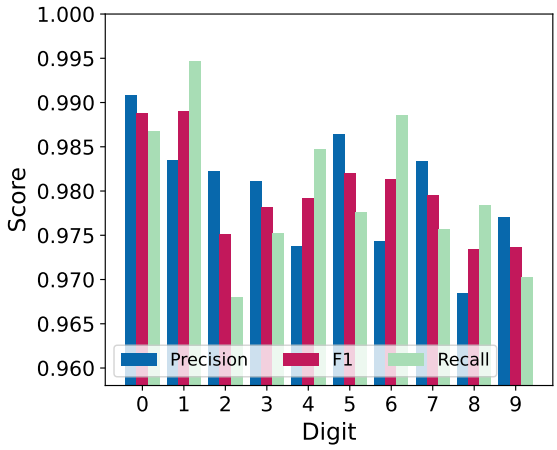


FIG. 9. Precision (blue), recall (light green) and F1 (red) per digit computed from the results of of our best performing model (Hyperparameters in table V) from the equations of 13, 14 and 15. Note that teh vertical axis starts at 0.960.

Appendix A: Use of LLMs and other assistance

A.1 Tools used. OpenAI Codex (GPT-6 Astra, November 2026)

A.2 Text writing and editing.

Section	Level	Notes
Abstract	0	...
Introduction	0	...
Theory / Methods	0	...
Results	0	...
Conclusion	0	...

A.3 Code generation and assistance.

File	Level	Description
Functions.py	0	Based on weekly exercises done in week 42
Functions_JAX.py	0	Copy of Functions.py with jax.numpy instead of numpy
Utils.py	0	Based on weekly exercises done in week 43
NeuralNetwork.py	1	Based on weekly exercises done in week 42, with LLM help for debugging
NeuralNetworkWrapper.py	1	Based on weekly exercises done in week 44, with LLM help for debugging
compute_gradient_test.py	2	Used LLM for help with computing gradient with JAX
NeuralNetwork_test.py	2	Used LLM for help with using NN with Scikit-learn
NeuralNetworkWrapper_test.py	0	Based on weekly exercises done in week 44
main_regression.py	3	Generated code for three plots and additional code for correct placement of legend and lables, and we adopted the code our existing loops
main_classification.ipynb	4	Got help, guidance and used code suggestions for every step in the file

Appendix B: Details of classification benchmarking with Scikit-Learn

In verifying our model, `MLPClassifer` was used to design a similar network. This was designed with one hidden layer with 256 nodes, with Adam as optimizer, a learning rate $\gamma = 10^{-3}$, minibatch size $M = 100$ and 10 epochs. Scikit-Learn has no implementation of the `Leaky ReLU`, so the standard `ReLU` was used in the hidden layer instead.

Scikit-Learn's model implements the L_2 norm using a different convention from our own. In our code, the penalty $\lambda\|W\|_2^2$ adds the term $2\lambda W$ to the gradient. In Scikit-Learn the penalty is $\frac{\alpha}{2M}\|W\|_2^2$, which adds the term $\frac{\alpha}{M}W$ to the gradient. Setting the two terms equal gives

$$\frac{\alpha}{M} = 2\lambda \Rightarrow \alpha = 2\lambda M$$

so that $\lambda = 10^{-4}$ corresponds to $\alpha = 0.02$. We also defined the same convergence parameter $\epsilon = 10^{-9}$ in Adam as in our own code. The number of epochs was fixed to 10 by setting `max_iter=10`, and early stopping was disabled (`tol=0`, `n_iter_no_change` set to larger than the number of epochs), so that both networks were trained for exactly the same number of epochs. Some differences still remains. The weight initialization, the shuffling of minibatches and the random numbers are not the same in the two codes. Some minor differences are therefore to be expected. For the logistic regression we made the comparison with Scikit-learn's `LogisticRegression` with default settings using the L-BFGS solver. This algorithm uses an L_2 penalty controlled by the inverse regularization strength $c = 1$. Since Scikit-learn sums the cross entropy over all n samples, this corresponds to $\lambda = 1/(2Cn) \approx 8 \cdot 10^{-6}$ in our notation. Compared to our value of $\lambda = 10^{-5}$ the biggest difference between the models are the optimization. Scikit uses a quasi-Newton method on the full dataset until convergence, while ours is trained for 10 epochs with Adam. [10]

Appendix C: Sample distribution

Table C1 is an overview of the distribution of samples per digit. Each digit makes up between 9% and 11.4% of the data, making them nearly balanced.

TABLE C1. Distribution of samples per digit [12]

Digit	Training	(%)	Test	(%)
0	5923	9.87	980	9.80
1	6742	11.24	1135	11.35
2	5958	9.93	1032	10.32
3	6131	10.22	1010	10.10
4	5842	9.74	982	9.82
5	5421	9.04	892	8.92
6	5918	9.86	958	9.58
7	6265	10.44	1028	10.28
8	5851	9.75	974	9.74
9	5949	9.91	1009	10.09
Sum	60000	100.00	10000	100.00

[1] Saint Augustine. *Confessions*. OUP Oxford, 2008. Written ca. 397–400 C.E.

[2] Warren S. McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, 5(4):115–133, December 1943.

[3] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 1958.

[4] Charu C. Aggarwal. *Neural Networks and Deep Learning: A Textbook*. Springer International Publishing, Cham, 2023.

[5] Su-Hyun Han, Ko Woon Kim, SangYun Kim, and Young Chul Youn. Artificial Neural Network: Understanding the Basic Concepts without Mathematics. *Dementia and Neurocognitive Disorders*, 17(3):83, 2018.

[6] Ajmal Rashid Bhat and Sumeer Ahmed. Artificial intelligence (AI) in drug design and discovery: A comprehensive review. *In Silico Research in Biomedicine*, 1:100049, January 2025.

[7] Noga Alon, Thomas F. Bloom, W. T. Gowers, Daniel Litt, Will Sawin, Arul Shankar, Jacob Tsimerman, Victor Wang, and Melanie Matchett Wood. Remarks on the disproof of the unit distance conjecture, 2026.

[8] OpenAI. Planar point sets with many unit distances. Technical report, OpenAI, May 2026.

[9] Morten Hjorth-Jensen. Applied data analysis and machine learning: Lecture notes. https://github.com/CompPhysics/MachineLearning, 2026.

[10] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[11] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[12] Yann LeCun, Corinna Cortes, and CJ Burges. Mnist handwritten digit database. *ATT Labs [Online]. Available: http://yann.lecun.com/exdb/mnist*, 2, 2010.

[13] Michael L. Waskom. seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60):3021, 2021.